SARAHA

Anonymous Messaging Platform

Full Project Documentation

Node.js · Express · MongoDB · Redis · JWT · Socket.io

Version 1.0 · March 2026

1. Project Overview

Saraha is a secure anonymous messaging platform that allows users to send hidden messages to registered users. Senders remain anonymous by default; however, if a sender has an account, the receiver may reply directly to them. The platform is built with a focus on privacy, security, and scalability.

1.1 Core Concept

App Name	Saraha — Anonymous Messaging Platform
Architecture	RESTful API (Node.js / Express)
Database	MongoDB with Mongoose ODM
Cache Layer	Redis (token revocation, OTP, rate limiting)
Auth Strategy	JWT (asymmetric RS256), OAuth 2.0 (Google, Facebook)
File Storage	Multer (local / cloud-ready)
Security Level	High — encrypted messages, hashed passwords, 2FA, rate limiting
Deployment	EvenNode / PM2 / Ngrok (dev SSL)

1.2 Key Features

- Send anonymous messages to any registered user
- Reply to messages if the sender has an account
- End-to-end message encryption (symmetric + asymmetric)
- Bcrypt + Argon2 hybrid password hashing
- JWT with refresh token rotation and revocation via Redis
- OAuth 2.0 social login (Google, Facebook)
- Email & phone OTP verification
- Two-Factor Authentication (2FA)
- Rate limiting, IP blocking, CORS, Helmet security headers
- Background jobs & queue for async processing
- Comprehensive logging with Morgan & Chalk

2. SDLC & Development Roadmap

The project follows an agile-inspired SDLC with clearly phased milestones. Each section below maps to a development sprint.

#	Phase	Deliverables
1	Foundation	Folder structure, User model, DB connection, Signup/Login
2	Security – Passwords	Hash (Bcrypt + Argon2 hybrid), Encryption (symmetric/asymmetric)
3	Tokens & Auth	Asymmetric JWT, Refresh tokens, Authentication middleware, Authorization
4	Social Login	Google OAuth 2.0 (FE + BE), Facebook OAuth 2.0
5	Validation	Joi schema validation (strings, numbers, arrays, dates, REGEX, files)
6	File Uploads	Multer (disk storage, validation, multi-file), Joi file validation
7	Caching & Tokens	Redis integration, token revocation (TTL), OTP storage
8	Email & OTP	NodeMailer, OTP send/resend/max-trial logic
9	Core Features	Forgot/Update password, Message model, Send/Get/Delete messages
10	Security Layer	CORS, Helmet, Rate limiting, IP blocking, Morgan, Chalk
11	Deployment	PM2, EvenNode, Ngrok SSL, GitHub CI

3. Folder Structure

The project uses a feature-based modular architecture to keep concerns separated and the codebase scalable.

```
saraha/
├── src/
│   ├── modules/
│   │   ├── user/
│   │   │   ├── user.model.js
│   │   │   ├── user.controller.js
│   │   │   ├── user.service.js
│   │   │   ├── user.router.js
│   │   │   └── user.validation.js
│   │   ├── message/
│   │   │   ├── message.model.js
│   │   │   ├── message.controller.js
│   │   │   ├── message.service.js
│   │   │   ├── message.router.js
│   │   │   └── message.validation.js
│   │   └── conversation/
│   │       ├── conversation.model.js
│   │       ├── conversation.controller.js
│   │       └── conversation.router.js
│   ├── middleware/
│   │   ├── auth.middleware.js
│   │   ├── validation.middleware.js
│   │   ├── rateLimit.middleware.js
│   │   └── upload.middleware.js
│   ├── utils/
│   │   ├── hash.util.js
│   │   ├── encryption.util.js
│   │   ├── token.util.js
│   │   ├── email.util.js
│   │   ├── otp.util.js
│   │   └── redis.util.js
│   ├── config/
│   │   ├── db.config.js
│   │   ├── redis.config.js
│   │   └── env.config.js
│   ├── repositories/
│   │   ├── user.repository.js
│   │   └── message.repository.js
│   └── app.js
├── uploads/
├── logs/
├── keys/
│   ├── private.key
│   └── public.key
├── .env
├── .gitignore
├── package.json
└── README.md
```

4. Data Models

4.1 User Model

The User model stores all account information and supports multiple authentication providers.

Field	Type	Required	Notes
firstName	String	Yes	Min 2, Max 50 characters
lastName	String	Yes	Min 2, Max 50 characters
email	String	Yes	Unique, lowercase, verified flag
password	String	Conditional	Hashed — not required for OAuth users
phone	String	No	Unique, E.164 format, phone verification flag
gender	Enum	No	male | female | other
provider	Enum	Yes	system | google | facebook
coverImage	String	No	URL / file path
profileImages	Array<String>	No	Multiple profile image URLs
address	Object	No	{ country, city, street, zip }
role	Enum	Yes	user | admin (default: user)
isDeleted	Boolean	Yes	Soft delete flag (default: false)
credentialChangedAt	Date	No	Invalidates old tokens on password change
twoFactorSecret	String	No	TOTP secret for 2FA

4.2 Message Model

Messages can contain text, images, videos, or emojis. All message bodies and attachments are encrypted at rest.

Field	Type	Required	Notes
body	String	Conditional	Encrypted text content (text | emoji)
attachment	String	Conditional	Encrypted file URL (image | video)
senderID	ObjectId	No	Null = anonymous sender
receiverID	ObjectId	Yes	Ref: User
conversationID	ObjectId	No	Ref: Conversation

Field	Type	Required	Notes
isDeleted	Boolean	Yes	Soft delete (default: false)
createdAt	Date	Auto	Mongoose timestamps

4.3 Conversation Model

A conversation is created when a receiver replies to an anonymous message from a registered sender.

Field	Type	Required	Notes
participants	Array<ObjectId>	Yes	Both sender and receiver user IDs
messages	Array<ObjectId>	Yes	Ref: Message
encryptedKey	String	Yes	Per-conversation encryption key
isDeleted	Boolean	Yes	Soft delete (default: false)

5. API Endpoints

5.1 User Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/signup	Public	Register new user (system)
POST	/api/auth/login	Public	Login & receive JWT + refresh token
GET	/api/auth/google	Public	Initiate Google OAuth 2.0 flow
GET	/api/auth/facebook	Public	Initiate Facebook OAuth 2.0 flow
POST	/api/auth/refresh	Public	Rotate refresh token
POST	/api/auth/logout	Bearer	Revoke token via Redis
POST	/api/auth/forgot-password	Public	Send OTP to email
POST	/api/auth/reset-password	OTP	Reset password with OTP token
PUT	/api/auth/change-password	Bearer	Change password (authenticated)
GET	/api/users/profile	Bearer	Get own profile
PUT	/api/users/profile	Bearer	Update profile fields
POST	/api/users/upload	Bearer	Upload profile/cover image
DELETE	/api/users/me	Bearer	Soft delete own account

5.2 Message Endpoints

Method	Endpoint	Auth	Description
POST	/api/messages	Optional	Send anonymous message to a user
GET	/api/messages/inbox	Bearer	Get all received messages
GET	/api/messages/:id	Bearer	Get single message by ID
DELETE	/api/messages/:id	Bearer	Soft delete a message

5.3 Conversation Endpoints

Method	Endpoint	Auth	Description
POST	/api/conversations	Bearer	Create or get existing conversation
GET	/api/conversations	Bearer	List all conversations
GET	/api/conversations/:id	Bearer	Get conversation with messages
DELETE	/api/conversations/:id	Bearer	Soft delete conversation

6. Security Architecture

6.1 Password Hashing — Hybrid Strategy

Saraha uses a hybrid hashing approach combining Bcrypt and Argon2 to leverage the strengths of both algorithms:

- Step 1 — Hash the password with Argon2id (memory-hard, resistant to GPU attacks)
- Step 2 — Hash the Argon2 output with Bcrypt (adds time-cost, widely trusted)
- This dual-layer approach means an attacker must break both algorithms to recover a password

6.2 Message Encryption

Algorithm	AES-256-GCM (symmetric) for message bodies
Key Exchange	RSA-OAEP (asymmetric) for encrypting the AES key
Per-message Key	Each message encrypted with a unique AES key
Key Storage	Encrypted AES key stored alongside the message
Conversation	Shared symmetric key, encrypted per participant

6.3 JWT Token Architecture

- Algorithm: RS256 (asymmetric — private key signs, public key verifies)
- Access Token: short-lived (15 min), contains user ID, role, device fingerprint
- Refresh Token: longer-lived (7 days), stored in Redis with TTL
- Token Revocation: tokens are added to a Redis blacklist on logout or password change
- credentialChangedAt field invalidates all tokens issued before a credential update
- Signature detection levels: detect token tampering and replay attacks

6.4 Two-Factor Authentication (2FA)

- TOTP (Time-based One-Time Passwords) using an authenticator app
- Secret stored encrypted in the user document
- Enabled/disabled via user profile settings
- Required on login when 2FA is active

6.5 Email & Phone Verification

Email OTP	6-digit OTP sent via NodeMailer, valid 10 min, max 3 resend attempts

Phone OTP	SMS OTP (via provider), same constraints
OTP Storage	Redis with TTL — auto-expires, no DB writes
Resend Logic	Exponential backoff after failed attempts

6.6 Request Security

- CORS: whitelist of allowed origins, credentials support
- Helmet: sets 15+ security HTTP headers (CSP, HSTS, X-Frame-Options, etc.)
- Rate Limiting: per-route limits with custom key generators (IP, user ID, path)
- IP Blocking: configurable blocklist, geo-blocking support
- Input Validation: Joi schemas on every request body, query, and params
- Multipart Validation: file type and size limits enforced via Multer + Joi

7. Course Curriculum Map

The following table maps all video lessons to their sections and corresponding project components.

Section 1 — Basics & Setup

#	Lesson	Covers
1	Folder Structure	Project layout, module separation
2	User Model	Mongoose schema, field types, virtuals
3	Signup & Login	Controller, service layer, basic auth flow
4	DB Repository	Repository pattern, Mongoose queries
5	Hash vs Encryption	Conceptual difference, when to use each
6	Hash Algorithm	SHA family, collision resistance, salting
7	Bcrypt	Bcrypt API, cost factor, compare
8	Bcrypt vs Argon2 Hybrid	Hybrid implementation for double protection
9	Encryption Concepts	Symmetric vs asymmetric encryption
10	Encryption Approach	Choosing algorithm, key management strategy
11	Generate Encryption	AES-256-GCM implementation
12	Generate Decryption	Decryption flow, IV handling, auth tags

Section 2 — Tokens & Authentication

#	Lesson	Covers
1	Asymmetric Token Concept	RS256, private/public key pairs
2	Sign vs Verify	jwt.sign, jwt.verify, error handling
3	Token Options	exp, iss, aud, sub, jti claims
4	Refresh Token	Rotation strategy, Redis TTL storage
5	Generate/Verify/Decode	Full token lifecycle implementation
6	Detect Signature Levels	Tamper detection, replay protection
7	Auth Middleware & Bearer	extracting Bearer token, middleware chain
8	Authorization	Role-based access control (RBAC)
9	Google OAuth (FE+BE)	Google Console setup, redirect URIs
10	Google OAuth (BE)	Passport.js / manual token exchange

Section 3 — Validation (Joi)

#	Lesson	Covers
1	validate vs validateAsync	Sync vs async validation, error messages
2	String Validation	min, max, email, uri, trim, lowercase
3	Number & Boolean	min, max, integer, truthy, falsy
4	Array & Date	items(), min/max length, date formats
5	Object Validation	keys(), unknown(), nested schemas
6	REGEX Validation	Custom patterns, password complexity
7	Shared Solutions	Reusable schemas, extend, concat
8	Share Profile	Profile update validation schema

Section 4 — File Uploads (Multer)

#	Lesson	Covers
0	Multer Intro	What is Multer, multipart/form-data basics
1	DiskStorage & FileName	Custom storage engine, filename strategy
2	Multer Folder Structure	Organizing upload paths by type
3	File Validation & Size	mimeType check, size limit enforcement
4	Multer Options	single(), array(), fields(), any(), none()
5	File Joi Validation	Validating uploaded file objects with Joi
6	Multipart Form Data	Converting FormData types, body parsing
7	Form-all & Revoke Token	Token TTL with mongoose-ttl pattern

Section 5 — Redis

#	Lesson	Covers
0	Redis Intro	In-memory store, use cases, data structures
1	String & Number	SET, GET, INCR, DECR, TTL
2	Hashes	HSET, HGET, HGETALL, HDEL
3	Lists	LPUSH, RPUSH, LRANGE, LLEN
4	Sets	SADD, SMEMBERS, SISMEMBER, SREM

#	Lesson	Covers
5	Connect Node-Redis	redis client, connection pooling, events
6	Redis Methods	Wrapper utilities for common patterns
7	Revoke Token	Token blacklist with automatic expiry

Section 6 — NodeMailer & OTP

#	Lesson	Covers
7	NodeMailer — Why & How	SMTP config, transporter setup, templates
8	Send OTP	OTP generation, email delivery, Redis storage
9	Resend OTP & Max Trial	Retry logic, max attempts, lockout

Section 7 — Password & Messages

#	Lesson	Covers
1	Forgot Password	OTP-based reset, secure flow
2	Update Password	Re-hash, invalidate tokens, credentialChangedAt
3	Postman Env & Scripts	Environment variables, pre-request scripts
4	Message Model Structure	Schema design, encryption hooks
5	Send Message	Anonymous flow, encrypt body, save
6	SenderID Logic	Conditional senderID, reply eligibility check
7	Get & Delete Message	Decrypt on read, soft delete
8	Router Options	Express Router config, prefix strategies

Section 8 — Security & Deployment

#	Lesson	Covers
1	CORS	Origin whitelist, preflight, credentials
3	Helmet	HTTP security headers overview
4	What Is Rate Limiting	Token bucket, sliding window concepts
5	Rate Limit Options	windowMs, max, handler, skip
6	Key Generator & Path	Per-user, per-IP, per-route limits
7	Block IP & Location	Blocklist middleware, geo restrictions

#	Lesson	Covers
8	GitHub & Deployment	Git flow, environment secrets, CI
9	Deployment Part 2 & PM2	PM2 cluster mode, ecosystem config
10	Ngrok Fake SSL	Local HTTPS tunneling for OAuth testing
11	Morgan	HTTP request logging, formats, streams
12	Chalk	Colorized console output, log levels
13	Deploy on EvenNode	PaaS deployment, env vars, MongoDB Atlas

8. Environment Configuration

Create a .env file in the project root. Never commit this file to version control.

```
# Server
PORT=3000
NODE_ENV=development

# MongoDB
MONGO_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/saraha

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=

# JWT Keys (RSA)
JWT_PRIVATE_KEY_PATH=./keys/private.key
JWT_PUBLIC_KEY_PATH=./keys/public.key
JWT_ACCESS_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d

# Encryption
ENCRYPTION_SECRET=<32-byte-hex-string>

# Email (NodeMailer)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your@email.com
SMTP_PASS=your_app_password

# Google OAuth
GOOGLE_CLIENT_ID=<your-google-client-id>
GOOGLE_CLIENT_SECRET=<your-google-client-secret>
GOOGLE_CALLBACK_URL=http://localhost:3000/api/auth/google/callback

# Facebook OAuth
FACEBOOK_APP_ID=<your-facebook-app-id>
FACEBOOK_APP_SECRET=<your-facebook-app-secret>
FACEBOOK_CALLBACK_URL=http://localhost:3000/api/auth/facebook/callback

# Rate Limiting
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX=100

# OTP
OTP_EXPIRY_SECONDS=600
OTP_MAX_ATTEMPTS=3
```

9. Non-Functional Requirements

9.1 Security

- Password hashing: Bcrypt + Argon2 hybrid (section 6.1)
- Message encryption: AES-256-GCM with RSA key wrapping
- Email + phone OTP verification on signup
- Two-Factor Authentication (TOTP) available for all users
- Joi validation on every incoming request
- Rate limiting: global + per-route with custom key generators
- IP blocklist middleware with geo-restriction support
- Helmet security headers + CORS whitelist
- Structured logging with Morgan (HTTP) and Chalk (console)

9.2 Scalability

- Stateless JWT authentication — horizontally scalable
- Redis for shared session/token state across instances
- MongoDB Atlas with connection pooling
- PM2 cluster mode — utilizes all CPU cores
- Background job queue for email sending and heavy processing
- File uploads ready for cloud storage (S3/Cloudinary swap)

9.3 Performance

- Redis caching for hot data (user profiles, token blacklist, OTPs)
- Database indexes on frequently queried fields (email, receiverID, isDeleted)
- Pagination on all list endpoints
- Lean queries where full Mongoose document not required

9.4 Availability

- PM2 with auto-restart on crash
- MongoDB Atlas with automatic failover
- Health check endpoint at GET /health
- Graceful shutdown handling (SIGTERM/SIGINT)

10. Getting Started

10.1 Prerequisites

- Node.js v18+
- MongoDB Atlas account (or local MongoDB 6+)
- Redis 7+ (local or Redis Cloud)
- Google Cloud Console project (for OAuth)
- Facebook Developer account (for OAuth)

10.2 Installation

```
# Clone the repository
git clone https://github.com/your-org/saraha.git
cd saraha

# Install dependencies
npm install

# Generate RSA key pair for JWT
mkdir keys
openssl genrsa -out keys/private.key 2048
openssl rsa -in keys/private.key -pubout -out keys/public.key

# Copy and fill in environment variables
cp .env.example .env

# Start in development mode
npm run dev

# Start in production with PM2
npm run build
pm2 start ecosystem.config.js
```

10.3 Running Tests

```
# Unit tests
npm test

# With coverage
npm run test:coverage

# Integration tests (requires running DB + Redis)
npm run test:integration
```

10.4 Deployment to EvenNode

1. Create an EvenNode account and new Node.js app

2. Set all environment variables in the EvenNode dashboard
3. Push code to EvenNode's Git remote: git push evennode main
4. Monitor logs via the EvenNode dashboard or PM2

Saraha — Anonymous Messaging Platform

Documentation generated with ♥ Node.js · Express · MongoDB · Redis